Regression Analysis - MIT 18.642

Table of contents

1 Lecture 6 - MIT 18.642- Regression Analysis **2**
- 1.1 Load data set Prostate . . . 2
- 1.2 Variables: . . . 3
- 1.3 Regression on Original Covariates . . . 4
- 1.4 Regression on Standardized Covariates . . . 5
- 1.5 R-squared . . . 6

2 Package MASS **7**
- 2.1 studres() in Package MASS . . . 7

3 Package car **8**
- 3.1 qqPlot() in Package car . . . 8
- 3.2 influencePlot() in Package car . . . 10
- 3.3 residualPlots() in Package car . . . 11
- 3.4 leveragePlots() in Package car . . . 12

4 Package STATS **13**
- 4.1 plot.lm() in STATS . . . 13

1 Lecture 6 - MIT 18.642- Regression Analysis

```r
# Load packages into R session (and install if necessary)

# require() returns TRUE if the package is successfully loaded
# and FALSE if it is not found.
if (!require("faraway")) install.packages("faraway")
if (!require("tidyverse")) install.packages("tidyverse")
library(faraway)
library(tidyverse)
library(car)
library(stats)
library(MASS) # For studres()
```

1.1 Load data set Prostate

```r
# Load the dataset into the environment
# Note: data() returns the dataset name,
# the actual values are stored in the 'prostate' dataframe
dataset_name <- data(prostate)
print(dataset_name)
```

```
[1] "prostate"
```

```r
# Display the first 3 observations of the prostate dataframe
print(head(prostate, 3))
```

```
      lcavol lweight age      lbph svi      lcp gleason pgg45     lpsa
1 -0.5798185  2.7695  50 -1.386294   0 -1.38629       6     0 -0.43078
2 -0.9942523  3.3196  58 -1.386294   0 -1.38629       6     0 -0.16252
3 -0.5108256  2.6912  74 -1.386294   0 -1.38629       7    20 -0.16252
```

1.2 Variables:

Table 1: Dataset Prostate (Library: faraway)

Variable	Description
lcavol	log cancer volume
lweight	log prostate weight
age	in years
lbph	log of the amount of benign prostatic hyperplasia
svi	seminal vesicle invasion
lcp	log of capsular penetration
gleason	a numeric vector
pgg45	percent of Gleason score 4 or 5
lpsa	response (log prostate specific antigen)

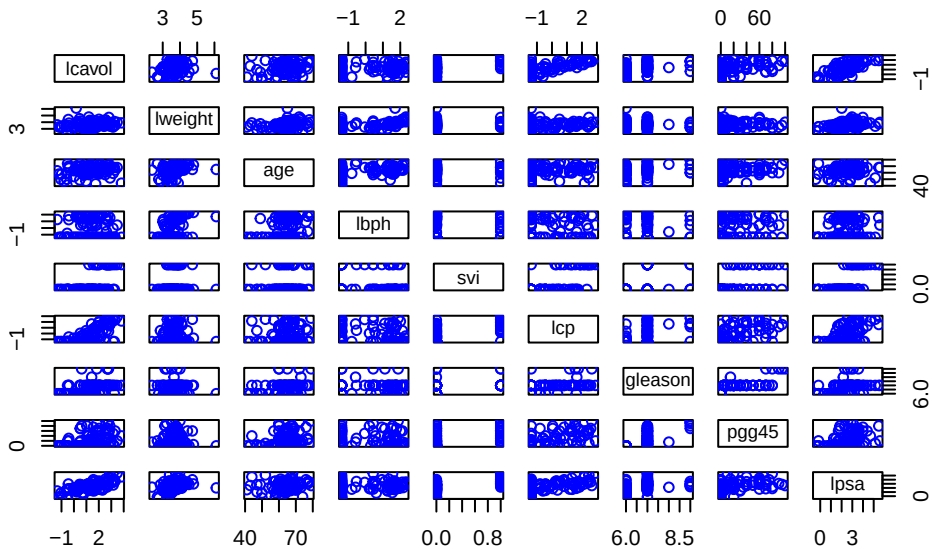
```r
round(apply(prostate,2,summary),digits=1)
```

```
        lcavol lweight  age lbph svi  lcp gleason pgg45 lpsa
Min.      -1.3     2.4 41.0 -1.4 0.0 -1.4     6.0   0.0 -0.4
1st Qu.    0.5     3.4 60.0 -1.4 0.0 -1.4     6.0   0.0  1.7
Median     1.4     3.6 65.0  0.3 0.0 -0.8     7.0  15.0  2.6
Mean       1.4     3.7 63.9  0.1 0.2 -0.2     6.8  24.4  2.5
3rd Qu.    2.1     3.9 68.0  1.6 0.0  1.2     7.0  40.0  3.1
Max.       3.8     6.1 79.0  2.3 1.0  2.9     9.0 100.0  5.6
```

```r
round(cor(prostate[,1:8]),digits=1)
```

```
        lcavol lweight age lbph  svi lcp gleason pgg45
lcavol     1.0     0.2 0.2  0.0  0.5 0.7     0.4   0.4
lweight    0.2     1.0 0.3  0.4  0.1 0.1     0.0   0.1
age        0.2     0.3 1.0  0.4  0.1 0.1     0.3   0.3
lbph       0.0     0.4 0.4  1.0 -0.1 0.0     0.1   0.1
svi        0.5     0.1 0.1 -0.1  1.0 0.7     0.3   0.5
lcp        0.7     0.1 0.1  0.0  0.7 1.0     0.5   0.6
gleason    0.4     0.0 0.3  0.1  0.3 0.5     1.0   0.8
pgg45      0.4     0.1 0.3  0.1  0.5 0.6     0.8   1.0
```

```r
pairs( prostate[,1:9], col="blue" )
```



1.3 Regression on Original Covariates

```
# Set a seed to ensure reproducibility
set.seed(1)

# Create a data frame with a single column
# containing the row indices of the prostate dataset
prostate_id <- data.frame(irow = 1:nrow(prostate))

# Randomly select 75% of the indices to be used for the training set
train_id <- slice_sample(prostate_id, prop = .75)

# Subset the original dataset to create the training set
train <- prostate[train_id$irow, ]

# Create the test set by excluding training indices (using the negative sign)
test <- prostate[-train_id$irow, ]

# Fit a linear regression model using all available predictors
# and summarize the results
```

```r
linear_regression <- lm(lpsa ~ ., data = train)
summary(linear_regression)
```

```

Call:
lm(formula = lpsa ~ ., data = train)

Residuals:
     Min       1Q   Median       3Q      Max 
-1.96222 -0.37818 -0.02121  0.42628  1.49784 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  0.501484   1.626900   0.308  0.75891    
lcavol       0.543473   0.104384   5.206 2.25e-06 ***
lweight      0.360912   0.195254   1.848  0.06924 .  
age         -0.009482   0.014579  -0.650  0.51779    
lbph         0.081743   0.072298   1.131  0.26249    
svi          0.876592   0.297203   2.949  0.00446 ** 
lcp         -0.042324   0.114845  -0.369  0.71371    
gleason      0.053957   0.195489   0.276  0.78344    
pgg45        0.003135   0.005241   0.598  0.55183    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7236 on 63 degrees of freedom
Multiple R-squared:  0.6298,	Adjusted R-squared:  0.5828 
F-statistic:  13.4 on 8 and 63 DF,  p-value: 4.198e-11
```

1.4 Regression on Standardized Covariates

```r
# Create a copy of the training set to avoid modifying the original data
train0 <- train

# Center and scale the first 8 predictor variables (mean=0, SD=1)
train0[, 1:8] <- scale(train[, 1:8])

# Fit the linear model on scaled data and display the summary results
summary(lm(lpsa ~ ., data = train0))
```

```
Call:
lm(formula = lpsa ~ ., data = train0)

Residuals:
     Min       1Q   Median       3Q      Max 
-1.96222 -0.37818 -0.02121  0.42628  1.49784 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  2.59101    0.08528  30.383  < 2e-16 ***
lcavol       0.60803    0.11678   5.206 2.25e-06 ***
lweight      0.18440    0.09976   1.848  0.06924 .  
age         -0.06348    0.09760  -0.650  0.51779    
lbph         0.11568    0.10231   1.131  0.26249    
svi          0.34937    0.11845   2.949  0.00446 ** 
lcp         -0.05591    0.15172  -0.369  0.71371    
gleason      0.03388    0.12273   0.276  0.78344    
pgg45        0.08027    0.13418   0.598  0.55183    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.7236 on 63 degrees of freedom
Multiple R-squared:  0.6298,	Adjusted R-squared:  0.5828 
F-statistic:  13.4 on 8 and 63 DF,  p-value: 4.198e-11
```

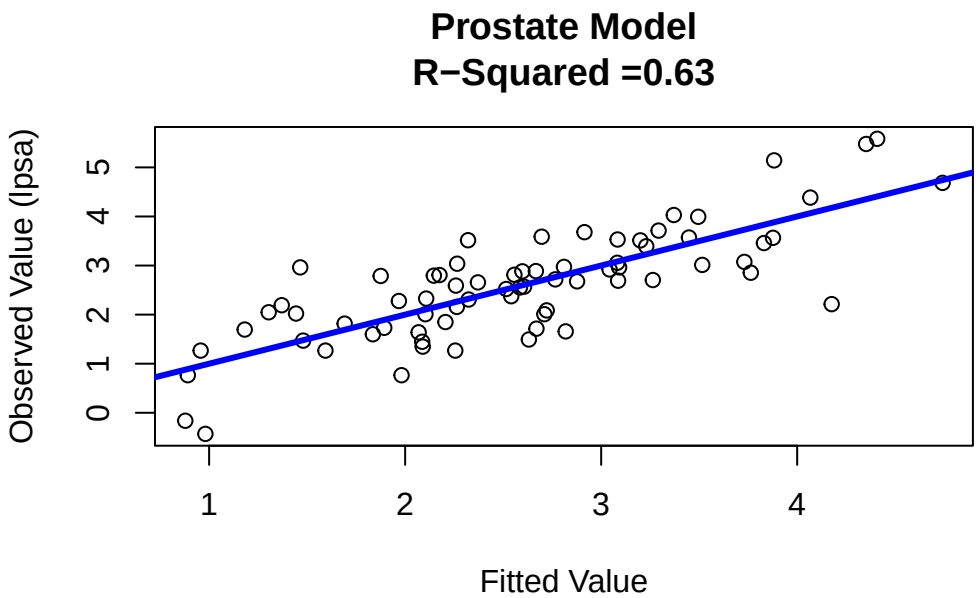
1.5 R-squared

```r
fit <- lm(lpsa~., data=train0)
fit.summary <- summary(fit)
names(fit.summary)
```

```
 [1] "call"          "terms"         "residuals"     "coefficients"  
 [5] "aliased"       "sigma"         "df"            "r.squared"     
 [9] "adj.r.squared" "fstatistic"    "cov.unscaled"  
```

```r
plot(fit$fitted.values,
     train0$lpsa,
     xlab="Fitted Value",
     ylab="Observed Value (lpsa)",
```

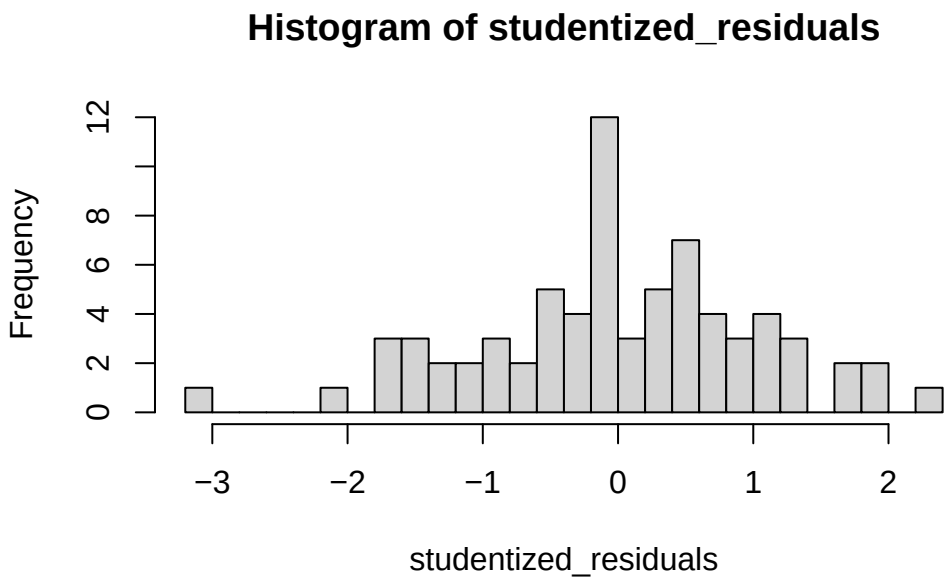
```
     main=paste0(
       c("Prostate Model",
         "\nR-Squared =",
         as.character(round(fit.summary$r.squared, digits=2))
         ),
       collapse = ""
     )
)
abline(lm(train0$lpsa ~ fit$fitted.values),col="blue",lwd=3)
```



2 Package MASS

2.1 studres() in Package MASS

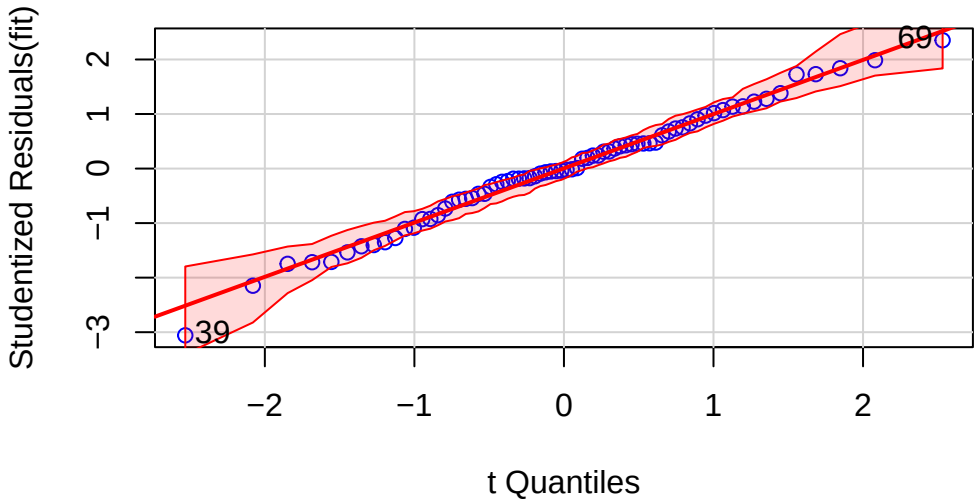
```
# Compute studentized residuals for the model
studentized_residuals <- studres(fit)
# Plot histogram of studentized residuals with 25 bins
hist(studentized_residuals, breaks = 25)
```



3 Package car

3.1 qqPlot() in Package car

```
# Create a Quantile-Quantile plot to check for normality
Extreme_Residuals <- qqPlot(fit, col = "blue", col.lines = "red")
```



The `qqPlot` function returns a **named integer vector** where the **values** represent the row positions of the most influential cases, and the **names** represent their corresponding row labels in the data set

```
# Construct the message using the positions in the training set
message <- paste0("The QQ-plot identifies the observations at training set",
                  " positions:\n [",
                  paste(as.numeric(Extreme_Residuals), collapse = ", "),
                  "]\n as potential outliers.")

# Print the final result
cat(message)
```

```
The QQ-plot identifies the observations at training set positions:
 [2, 65]
 as potential outliers.
```

```
# Option A: Using the labels (The safest way)
outlier_data <- train0[names(Extreme_Residuals), ]
print(outlier_data)
```

```
      lcavol   lweight       age       lbph        svi        lcp   gleason
```

```
39  1.158361 0.8205313 0.5933575  0.8196158  2.0212168  1.5392854  0.420319
69 -1.618974 1.4535083 0.7427342 -1.1307808 -0.4878799 -0.8973351 -1.172469
        pgg45    lpsa
39  0.4719599 2.21375
69 -0.8950963 2.96269
```

```r
# Option B: Using the values (Position in the training set)
outlier_data_alt <- train0[Extreme_Residuals, ]
print(outlier_data_alt)
```

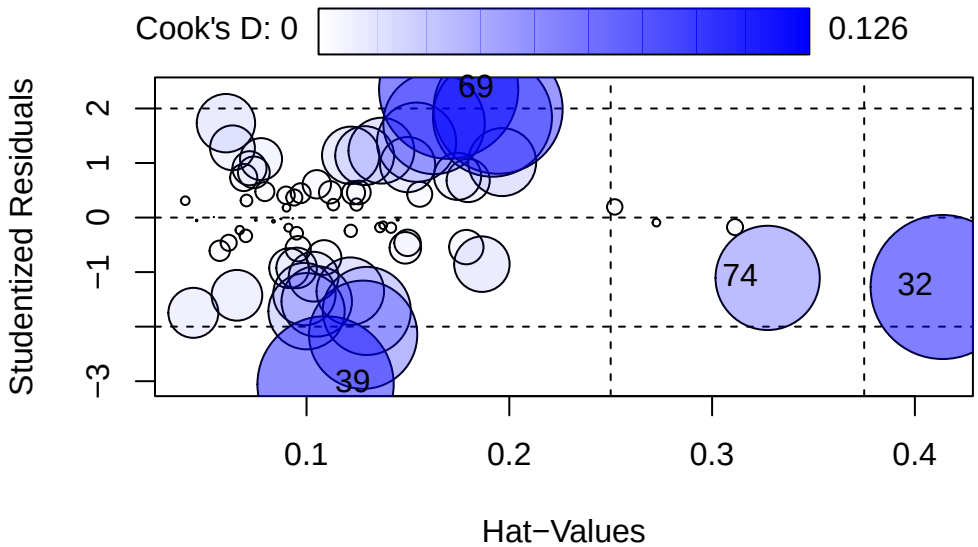
```
      lcavol   lweight       age       lbph        svi        lcp   gleason
39  1.158361 0.8205313 0.5933575  0.8196158  2.0212168  1.5392854  0.420319
69 -1.618974 1.4535083 0.7427342 -1.1307808 -0.4878799 -0.8973351 -1.172469
        pgg45    lpsa
39  0.4719599 2.21375
69 -0.8950963 2.96269
```

3.2 influencePlot() in Package car

Unlike `qqPlot` (which only identifies points with extreme residuals), `influencePlot` is more rigorous because it looks for observations that meet any of the following conditions:

- **High Studentized Residuals (Outliers):** Observations where the model fails significantly to predict the response.
- **High Leverage:** Points with extreme values in the predictor variables (X) that have the potential to pull the regression line toward them.
- **High Global Impact (Cook's Distance):** Observations that, if removed, would substantially change the model's coefficients.

```r
# Store the influence diagnostics in a data frame
influence_points <- influencePlot(lm(lpsa ~ ., data = train0))
```




```
# Display the coordinates and metrics of the most influential cases
print(influence_points)
```

```
     StudRes       Hat      CookD
39 -3.057931 0.1093825 0.11267049
74 -1.106432 0.3273886 0.06597263
32 -1.273374 0.4136773 0.12587260
69  2.352556 0.1701000 0.11757924
```

```
# To access the specific row names of these influential points
influential_names <- rownames(influence_points)
influential_names
```

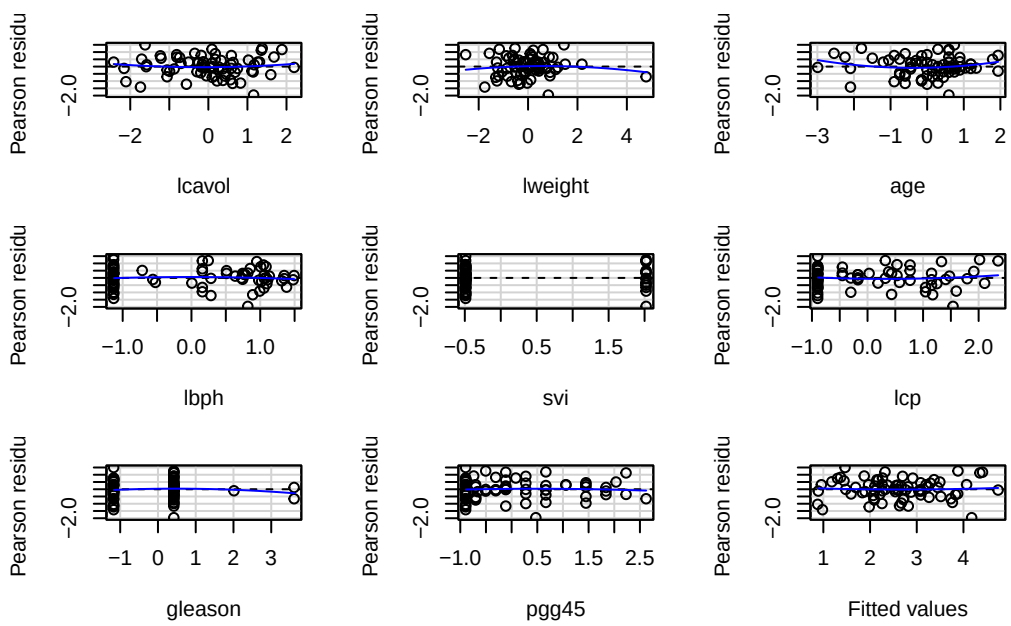
```
[1] "39" "74" "32" "69"
```

3.3 residualPlots() in Package car

```
residualPlots(lm(lpsa~., data=train0),tests=TRUE,plot=FALSE)
```

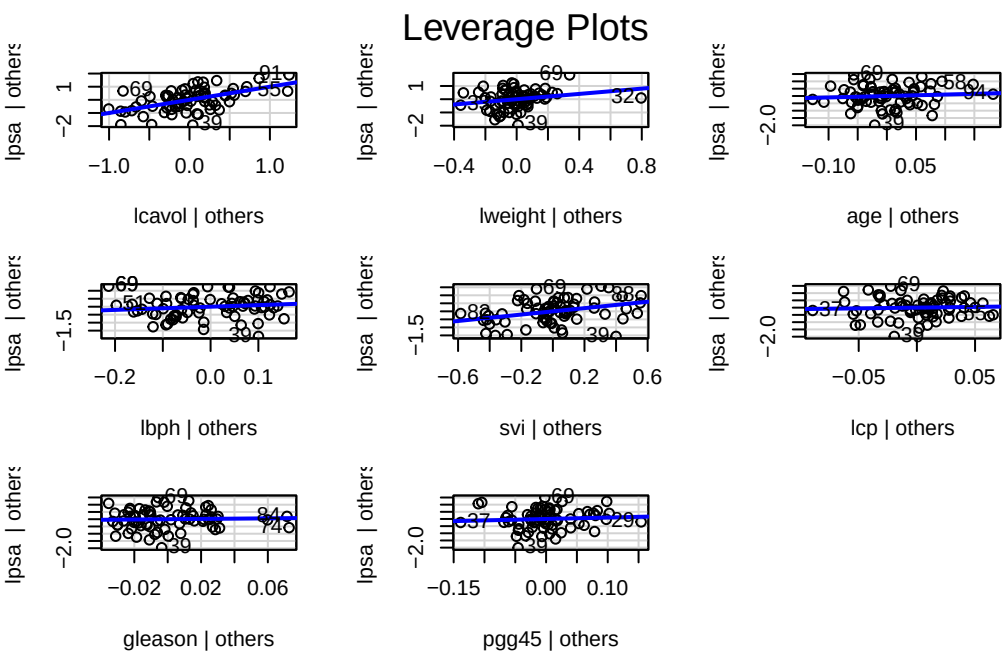
```
           Test stat Pr(>|Test stat|)
lcavol        0.7273           0.4698
lweight      -0.7810           0.4377
age           1.3759           0.1738
lbph         -0.4307           0.6681
svi          -1.7301           0.0886 .
lcp           0.6207           0.5371
gleason      -0.7582           0.4512
pgg45        -0.3076           0.7594
Tukey test    0.3931           0.6942
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
residualPlots(lm(lpsa~., data=train0),tests=FALSE,plot=TRUE)
```



3.4 leveragePlots() in Package car

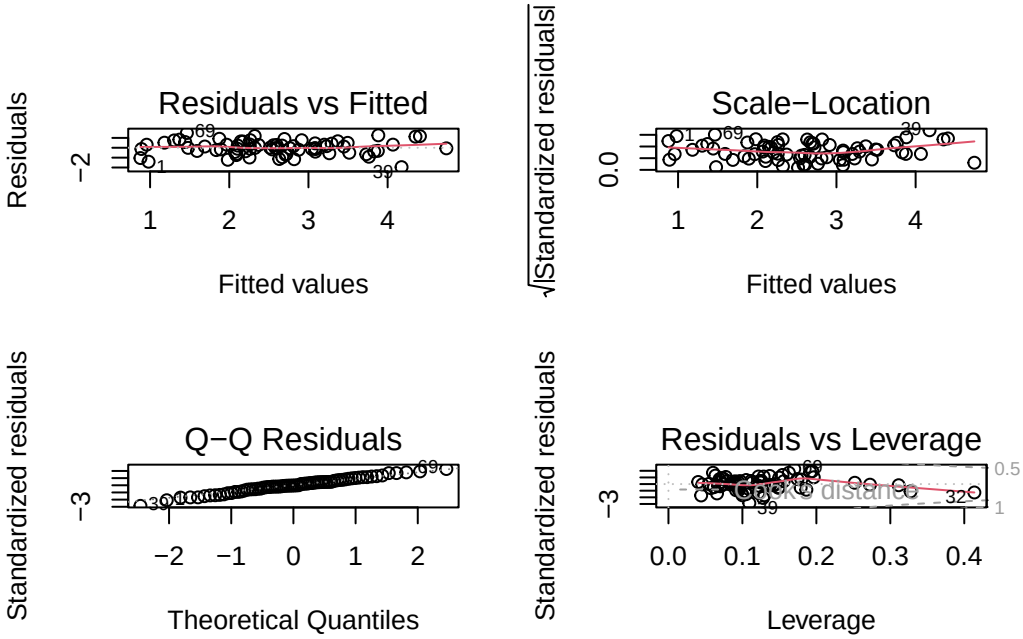
```r
leveragePlots(lm(lpsa~., data=train0))
```



4 Package STATS

4.1 plot.lm() in STATS

```
# The R function $plot.lm()$ generates a 2x2 display of plots
oldpar=par(no.readonly=TRUE)
layout(matrix(c(1,2,3,4),2,2)) # optional 4 graphs/page
plot(fit)
```




```
par(oldpar,no.readonly=TRUE)
```